Assignment 1: Epidemic rate equations

Practical information

Deadline: Saturday 7/2 23.59

Resources:

- ERDA for file storage
- Jupyter for the Terminal to access DAG
- DAG for testing, visualization, and benchmarks

Hand-in:

- A report of up to 3 pages in length (excluding the code)
- Your code both as source file as well as in pdf
- An up to 3-minute video where one person from the group scrolls through the code and explains what you have done.
- Use the template on Absalon to include your code in the report

Introduction

A simple mathematical description of the spread of a disease in a population is the so-called SIR model, which divides the (fixed) population of *N* individuals into three ”compartments” that may vary as a function of time, *t*:

- *S(t)* are those susceptible but not yet infected with the disease.
- *I(t)* are the number of infectious individuals. These are infected and can infect others.
- *R(t)* are those individuals who have recovered from the disease and now have immunity to it.

The SIR model describes the change in the population of each of these compartments in terms of two parameters, β and γ. β describes how infectious the disease is: on average, in absence of immunity, every infected person infects β others. This rate is reduced by the fraction of the population that is susceptible (i.e. are not immune yet), S/N. γ is the mean recovery rate: That is, 1/γ is the mean period of time during which an infected individual can pass it on.

The differential equations describing this model were first derived by Kermack and McKendrick [Proc. R. Soc. A, 115, 772 (1927)]. The inspiration from this example come from Chris Hill, Learning Scientific Programming with Python. Here are the equations:

$$\frac{dS}{dt} = -\beta I \frac{S}{N}$$

$$\frac{dI}{dt} = \beta I \frac{S}{N} - \gamma I$$

$$\frac{dR}{dt} = \gamma I$$

How to use the code templates

For this tutorial we need g++ and python, which is provided on ERDA. Spin up a Jupyter session on DAG. Important! Select the “HPC notebook” notebook image. This is the image we use for the course. In the launcher, launch the Terminal.

In the terminal (or the folder view on the right side) you can see two folders: `erda_mount` and `hpc_course`. `erda_mount` contains your own storage area. This is where you save files. In the

folder `hpc_course` the exercises are stored. If you have not done so already, start by making a new folder in your storage area for the course. Then copy the exercise to the folder and enter into the folder. You can write '`ls`' to get a file listing of the folder.

```
cd erda_mount
mkdir HPPC
cd HPPC
cp -a ~/hpc_course/module1 .
cd module1
ls
```

To be able to edit the files for the exercise navigate to the same folder in the file view. Here you can open four files:

- Makefile
- SIR.cpp
- SIR_python.cpp
- Unknown_SIR_data.csv

Task 1: Implement and validate the SIR model (4 points)

Write a C++ code which integrates these equations based on SIR.cpp. Every place that has to be modified has been marked with "TODO". Show the correctness by checking that your code does the right thing in at least 4 different limits. This can for example be the evolution after 1 step in different limits of $\beta, \gamma, \beta/\gamma$, initial conditions, whether it reaches a steady state, timescale for reaching a steady state or whatever you find appropriate. Argue that the result of your code is correct e.g. by comparing with analytical results.

To complete this task, you have to do two things:

- Submit your code both as source as well as in pdf
- Create and upload an up to three-minute video done by one group member, who explains the code (e.g. briefly what it does in different part of the source code), and flip over to python plots and argue that the results are correct.

The TA has to look through ≈20 of these videos, and it is therefore important that you keep the 3 minute limit, to make it feasible.

Task 2: Use and plot the SIR model (2 points)

Simulate an epidemic with parameters $\beta = 0.2, \gamma = (10\ days)^{-1}$ spreading in a population of 1000 people. For initial parameters of *S*, *I* and *R*, think about how a population starts. Save the data into a file, and load and plot it in python, showing *S*, *I* and *R* on the same plot as a function of time *t*. Discuss how you chose the timestep Δt. Plot both a simulation with a high Δt and one with so low Δt that the plot has steps.

Task 3: Timestep convergence (4 points)

We want to explore more systematically how different step sizes affect the accuracy of the numerical solution, and what is sufficient. We quantify the difference between a solution integrated with timestep Δt and timestep $\frac{\Delta t}{2}$ as their maximal disagreement, that is

$$err = \max_t \{S^{\Delta t}(t) - S^{\frac{\Delta t}{2}}(t), I^{\Delta t}(t) - I^{\frac{\Delta t}{2}}(t), R^{\Delta t}(t) - R^{\frac{\Delta t}{2}}(t)\}$$

where e.g. $S^{\Delta t}(t)$ is the number of susceptible people at time t in a simulation with timestep Δt. To make comparison of models with different timesteps feasible, you must implement a

frequency with which you write out t, S, I, R to a file. If this frequency of writing data to a file is divisible by all timestep sizes, the different output files will contain the same number of records, and data at exactly the same times. By writing out "t", the time in each record in the file, you can validate that this is the case.

With the same (β, γ) as before

- calculate err and plot it as a function of the timestep size Δt for at least 5 timestep sizes in the range 1 day to 1e-6 days.
- Make a plot of err as a function of time for both a large and a small Δt.

Discuss your results.

Task 4 (Optional!!!): Integrate C++ with Python. (up to 2 extra points)

This is an exercise for the interested, showing how to integrate C++ with python. This can be very useful to squeeze out performance from Python in some cases e.g. for Monte Carlo sampling of expensive functions, or when the algorithm has to be written in terms of for-loops. Only do this if you find it interesting (the time/point cost benefit is bad :)) and the total number of points is still capped at 10.

Sometimes, C++ (and other low-level languages such as Fortran, C, Rust) is used because for large projects, they are better suited than e.g. python. Other times, we use it to speed up the 1% of our code which takes 99% of the execution time, meaning that we can continue to use python for convenience. This could for instance be the log-likelihood in a Monte Carlo sampler or the integrand in a SciPy quad integral. This is how NumPy manages to be often orders of magnitudes faster than the equivalent python code. In this case we need to "bind" the two languages together. This exercise demonstrates this.

Rather than running the C++ code, saving a file and then loading and plotting in python, you are now tasked with calling the C++ code as a function inside python. For this, use the cpp file SIR_python.cpp that uses pybind11++ library, which can be installed with pip install pybind11. If you try to run this on your own pc, you might also need apt-get python3-dev if you are on Linux.

After implementing the SIR model, you should be able to compile it by uncommenting all code in the Makefile (remove all "#") and then calling make. Afterwards, you should be able to import and run the C++ code from e.g. a jupyter notebook like:

```
import SIR_python

SIR_python.integrate_system(S,I,R,beta, gamma, dt, n_times, return_every)
```

Using this, fit the (slightly noisy) data in Unknown_SIR_data.csv. This data has S0=0, I0=1 and R0=0, but determining beta and gamma is up to you! **Show your fitted values and your fit on the data**.

Note that for task 4 you must restart the kernel to reload a new version after it has been compiled.